



Pontificia Universidad Católica de Chile  
Departamento de Ciencias de la Computación

Diplomado en Inteligencia Artificial

## Clase 18

Modelos de lenguaje, Word2Vec, GloVe y SkipThought

Pablo Messina



**IA** Lab



# ¿Qué estudiaremos en este curso?

## Clase 1

### **Introducción a NLP**

¿Qué es y por qué es una tarea difícil?

Aplicaciones

Técnicas clásicas

Herramientas concretas

## Clase 2

### **Modelos de lenguaje**

Modelos de Lenguaje

Representaciones Discretas v/s Continuas (Distribuidas)

W2V, GloVe, SkipThought

## Clase 3

### **Modelos modernos**

ELMo, GPT y BERT

Librería para usar estos modelos

Aplicaciones concretas

¿Qué estudiaremos en este curso?

Clase 4

**Aplicaciones:**

Generación de Resúmenes

Clase 5

**Aplicaciones:**

Modelos de Pregunta-Respuesta

# Contenidos de la clase de hoy

1

## Modelos de Lenguaje (LM)

¿Qué son los modelos de lenguaje?

¿Para qué sirven?

Modelos de N-gramas

2

## Representaciones Discretas v/s Continuas (Distribuídas)

Representaciones Discretas

Representaciones Continuas (Distribuídas)

3

## Ejemplos concretos

Word2Vec

GloVe

SkipThought

Trabajo práctico

# Contenidos de la clase de hoy

1

## Modelos de Lenguaje (LM)

¿Qué son los modelos de lenguaje?

¿Para qué sirven?

Modelos de N-gramas

2

## Representaciones Discretas v/s Continuas (Distribuídas)

Representaciones Discretas

Representaciones Continuas (Distribuídas)

3

## Ejemplos concretos

Word2Vec

GloVe

SkipThought

Trabajo práctico

## Modelos de Lenguaje ¿Qué son?

Nos permiten estimar la probabilidad de un texto (secuencia de palabras)

$$P(\text{Hola}) = 0.1$$

$$P(\text{Hola, cómo estás?}) = 0.05$$

$$P(\text{Qué bonito está el día}) = 0.02$$

$$P(\text{Se me atrofió el esternocleidomastoideo}) = 0.00001$$

$$P(\text{supernova flor barroco saltar hola chao}) = 0.00000000001$$

**Observación:** ciertas secuencias de palabras son más probables que otras

## Modelos de Lenguaje: Jugando con la distribución P

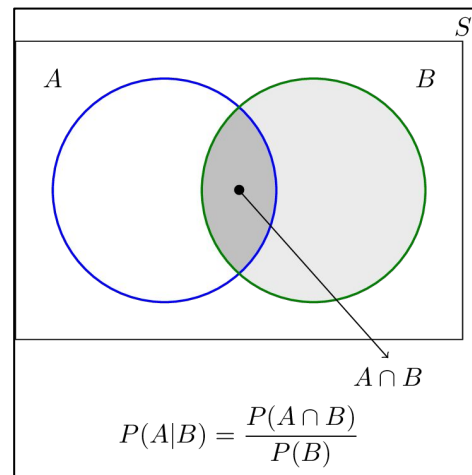
Podemos considerar también probabilidades **condicionales**, por ejemplo, la probabilidad de un texto dado un contexto previo

$$P(\text{perro}) = 0.01$$

$$P(\text{perro}|\text{un}) = 0.03$$

$$P(\text{perro}|\text{mordió un}) = 0.05$$

$$P(\text{perro}|\text{Me mordió un}) = 0.1$$



**Observación:** tener información del contexto puede alterar la distribución de probabilidad, haciendo ciertas palabras más probables que otras.

## Modelos de Lenguaje: Jugando con la distribución P

**Regla de la cadena (probabilidades):** podemos calcular la probabilidad  $P(\text{texto})$  condicionando palabra a palabra:

$$\begin{aligned}P(A B C D) &= \\P(D \mid A B C) * P(A B C) &= \\P(D \mid A B C) * P(C \mid A B) * P(A B) &= \\P(D \mid A B C) * P(C \mid A B) * P(B \mid A) * P(A) &= \\P(A) * P(B \mid A) * P(C \mid A B) * P(D \mid A B C)\end{aligned}$$

$$P(\text{hola cómo estás}) = P(\text{hola}) * P(\text{cómo} \mid \text{hola}) * P(\text{estás} \mid \text{hola cómo})$$

**Observación:** calcular probabilidades de secuencias largas se puede hacer con un modelo que calcule probabilidades de palabras individuales dado un contexto



## Modelos de Lenguaje ¿De qué me sirve todo esto?

1) **Natural Language Generation:** dado un texto inicial  $X$ , con nuestro modelo de lenguaje  $P$  podemos escoger la siguiente palabra  $w$  que maximice  $P(w | X)$  y así sucesivamente:

*Texto inicial:  $X$*

*Buscamos  $w_0$  que maximice  $P(w_0 | X)$*

*Buscamos  $w_1$  que maximice  $P(w_1 | X w_0)$*

*Buscamos  $w_2$  que maximice  $P(w_2 | X w_0 w_1)$*

*Buscamos  $w_3$  que maximice  $P(w_3 | X w_0 w_1 w_2)$*

*Buscamos  $w_4$  que maximice  $P(w_4 | X w_0 w_1 w_2 w_3)$*

*Texto final:  $X w_0 w_1 w_2 w_3 w_4$*

## Modelos de Lenguaje ¿De qué me sirve todo esto?

2) **Machine Translation:** dado un texto inicial  $X$  en lenguaje de origen, queremos generar un texto de salida  $Y$  en lenguaje de destino que maximice  $P(Y | X)$

*Texto inicial:  $X$  (en inglés)*

*Buscamos  $w_0$  que maximice  $P(w_0 | X)$*

*Buscamos  $w_1$  que maximice  $P(w_1 | X, w_0)$*

*Buscamos  $w_2$  que maximice  $P(w_2 | X, w_0 w_1)$*

*Buscamos  $w_3$  que maximice  $P(w_3 | X, w_0 w_1 w_2)$*

*Buscamos  $w_4$  que maximice  $P(w_4 | X, w_0 w_1 w_2 w_3)$*

*Traducción:  $Y = w_0 w_1 w_2 w_3 w_4$  (en español)*

## **Modelos de Lenguaje** ¿De qué me sirve todo esto?

### **3) Otras aplicaciones:**

- Spelling correction
- Document Summarization
- Question answering
- Sentence completion
- Speech recognition
- Information retrieval
- Code completion
- etc.

## Modelos de Lenguaje Caso particular: N-gramas

Algunos Modelos de Lenguaje acotan el contexto de la probabilidad condicional a una ventana de a lo más N palabras:

$$\text{Con } N = 0: \quad P(w_0 w_1 w_2 w_3) = P(w_0) * P(w_1) * P(w_2) * P(w_3)$$

$$\text{Con } N = 1: \quad P(w_0 w_1 w_2 w_3) = P(w_0) * P(w_1|w_0) * P(w_2|w_1) * P(w_3|w_2)$$

$$\text{Con } N = 2: \quad P(w_0 w_1 w_2 w_3) = P(w_0) * P(w_1|w_0) * P(w_2|w_0 w_1) * P(w_3|w_1 w_2)$$

$$\text{Con } N = 3: \quad P(w_0 w_1 w_2 w_3) = P(w_0) * P(w_1|w_0) * P(w_2|w_0 w_1) * P(w_3|w_0 w_1 w_2)$$

**Observación:** dependiendo del modelo, acotar el contexto puede reducir el costo computacional de calcular la probabilidad, aunque puede bajar el desempeño debido a pérdida de información

# Contenidos de la clase de hoy

1

## Modelos de Lenguaje (LM)

¿Qué son los modelos de lenguaje?

¿Para qué sirven?

Modelos de N-gramas

2

## Representaciones Discretas v/s Continuas (Distribuídas)

Representaciones Discretas

Representaciones Continuas (Distribuídas)

3

## Ejemplos concretos

Word2Vec

GloVe

SkipThought

Trabajo práctico

## Modelos de Lenguaje Representaciones Discretas vs. Continuas

**Problema:** se busca resolver dos cosas:

- 1) Cómo vamos a **representar las palabras**.
- 2) Cómo vamos a calcular la **probabilidad P** de nuestro modelo de lenguaje.

Dos estrategias generales:

- |                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>1) <u>Approach discreto</u>:</p> <ul style="list-style-type: none"><li>- Representar palabras con IDs</li><li>- Calcular la probabilidad P usando n-gramas y estadísticas discretas</li></ul> | <p>2) <u>Approach continuo (machine learning)</u>:</p> <ul style="list-style-type: none"><li>- Representar palabras con vectores distribuidos semánticamente en un espacio continuo (<b>a.k.a. word embeddings, se aprenden</b>)</li><li>- Calcular la probabilidad P usando redes neuronales (<b>pesos de la red se aprenden</b>)</li></ul> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## **Modelos de Lenguaje** Representaciones Discretas

$P(\text{the cat sat on the mat}) = ?$

## Modelos de Lenguaje Representaciones Discretas

$P(\text{the cat sat on the mat}) =$

$P(\text{the}) * P(\text{cat}|\text{the}) * P(\text{sat}|\text{cat}) * P(\text{on}|\text{sat}) * P(\text{the}|\text{on}) * P(\text{mat}|\text{the})$

Contextos son 1-gramas





## Modelos de Lenguaje Representaciones Discretas

$P(\text{the cat sat on the mat}) =$

$P(\text{the}) * P(\text{cat}|\text{the}) * P(\text{sat}|\text{cat}) * P(\text{on}|\text{sat}) * P(\text{the}|\text{on}) * P(\text{mat}|\text{the})$

Contextos son 1-gramas



¿Cómo podemos calcular  $P(X|C)$ ?

Contando cuántas veces la secuencia “C X” aparece en un **corpus** (dataset) de documentos (textos), y dividiendo por la cantidad de veces que “C” aparece en el mismo corpus. Es decir:

$P(X|C) = \text{count\_in\_corpus}(\text{“C X”}) / \text{count\_in\_corpus}(\text{“C”})$

## Modelos de Lenguaje Representaciones Discretas

| ID | Contenido del Corpus     |
|----|--------------------------|
| S1 | the cat sat on the mat   |
| S2 | the dog sat on the cat   |
| S3 | the cat caught the mouse |

**Vocabulario (V):** {the, cat, sat, on, mat, dog, caught, mouse}

## Modelos de Lenguaje Representaciones Discretas

| ID | Contenido del Corpus     |
|----|--------------------------|
| S1 | the cat sat on the mat   |
| S2 | the dog sat on the cat   |
| S3 | the cat caught the mouse |

**Vocabulario (V):** {the, cat, sat, on, mat, dog, caught, mouse}

$$P(\text{cat}|\text{the}) = \frac{\text{count\_in\_corpus}(\text{"the cat"})}{\text{count\_in\_corpus}(\text{"the"})}$$

## Modelos de Lenguaje Representaciones Discretas

| ID | Contenido del Corpus                          |
|----|-----------------------------------------------|
| S1 | <u>the</u> <u>cat</u> sat on <u>the</u> mat   |
| S2 | <u>the</u> dog sat on <u>the</u> <u>cat</u>   |
| S3 | <u>the</u> <u>cat</u> caught <u>the</u> mouse |

**Vocabulario (V):** {the, cat, sat, on, mat, dog, caught, mouse}

$$P(\text{cat}|\text{the}) = \frac{\text{count\_in\_corpus}(\text{"the cat"})}{\text{count\_in\_corpus}(\text{"the"})}$$

$$3 / 6 = 0.5$$

## **Modelos de Lenguaje** Representaciones Discretas

### Limitaciones:

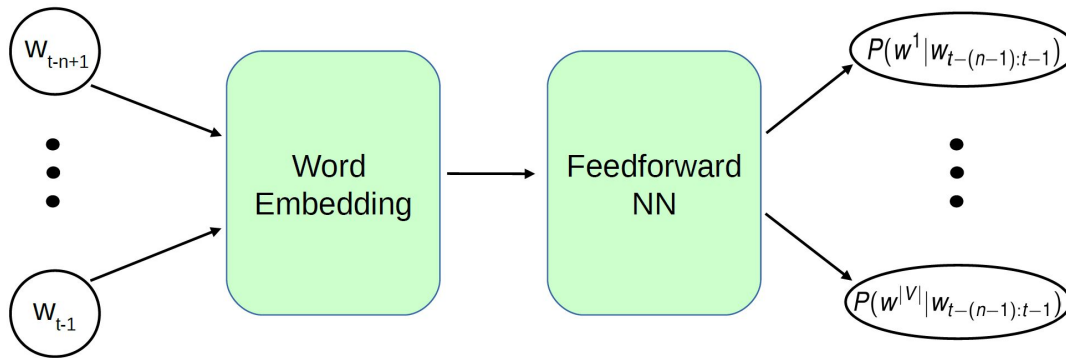
- Las palabras se representan con IDs, lo cual carece de información semántica.
- Está limitado a n-gramas, donde n generalmente no puede ser muy grande (por ej.  $n = 2, 3, 4$  o  $5$ ), ya que para n más grandes los n-gramas se vuelven demasiado infrecuentes (entre más grande el n, más improbable es que una combinación de n palabras haya aparecido en el corpus de entrenamiento).
- Por el punto anterior, estos métodos no son capaces de generalizar a combinaciones de n-gramas más grandes o que nunca fueron vistos en el corpus de entrenamiento (ya que la frecuencia asociada es 0).

## Modelos de Lenguaje Representaciones Continuas (Distribuidas)

### A.k.a. Neural Probabilistic Language Models

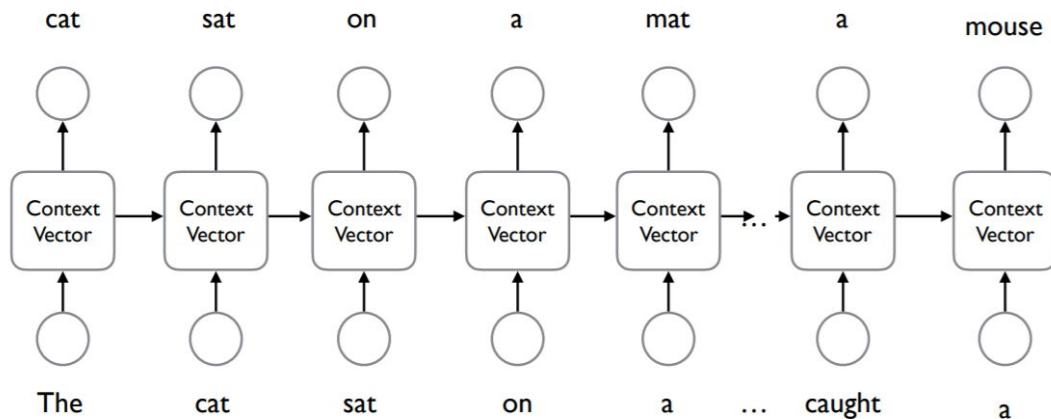
Objetivo 1: Aprender una representación continua distribuida para cada palabra  $w$  en el vocabulario, i.e., **aprender un word embedding**.

Objetivo 2: Aprender los pesos de una red neuronal que prediga la probabilidad de una palabra  $w$  dado un contexto  $h(w)$ , i.e., **aprender  $P(w | h(w))$**



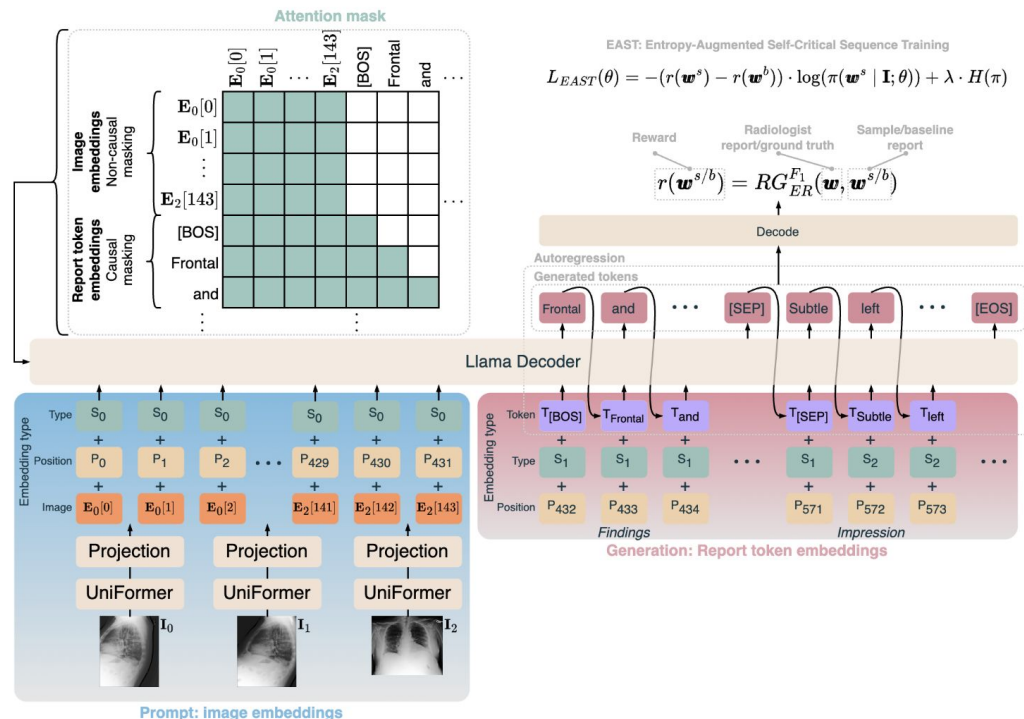
## Modelos de Lenguaje Representaciones Continuas (Distribuidas)

No es obligatorio que la red neuronal usada sea una feed forward. Otra opción puede ser una red neuronal recurrente (RNN):



# Modelos de Lenguaje Representaciones Continuas (Distribuidas)

O incluso un  
Transformer  
multimodal:

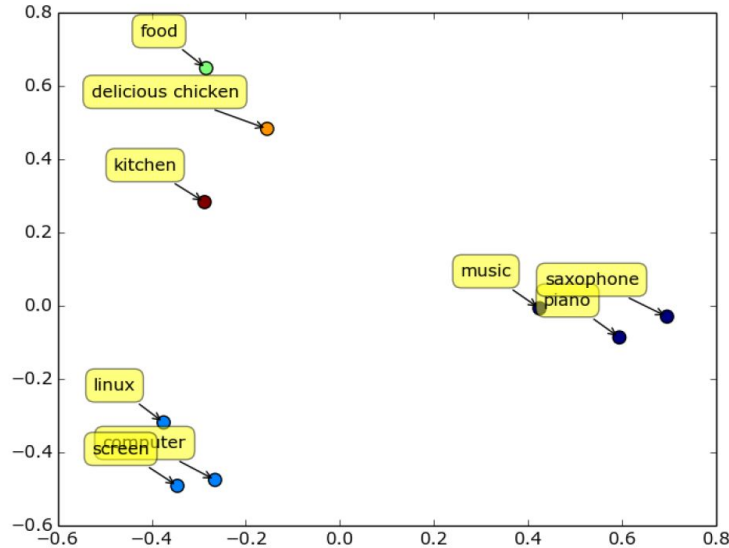




## Modelos de Lenguaje Representaciones Continuas (Distribuidas)

Ventajas:

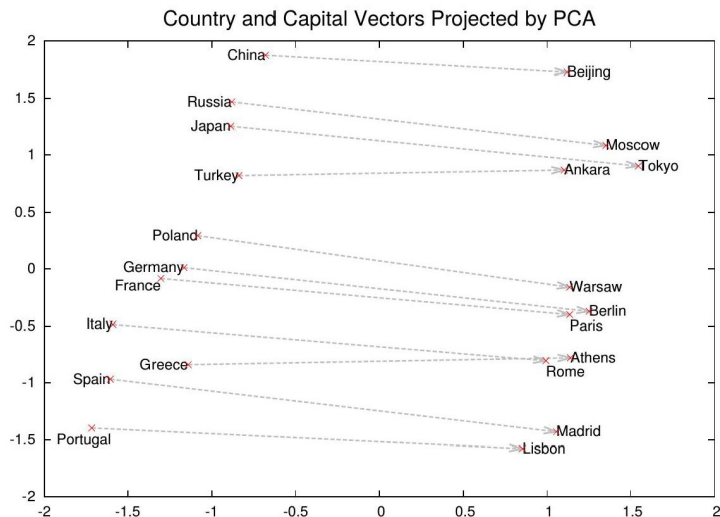
- 1) Cercanía semántica: Palabras que suelen aparecer en contextos similares adquieren representaciones similares (sus vectores quedan más cerca entre sí)



## Modelos de Lenguaje Representaciones Continuas (Distribuidas)

Ventajas:

- 2) Composicionalidad aditiva: los word embeddings se distribuyen de manera tal que a veces es posible hacer operaciones algebraicas para obtener analogías



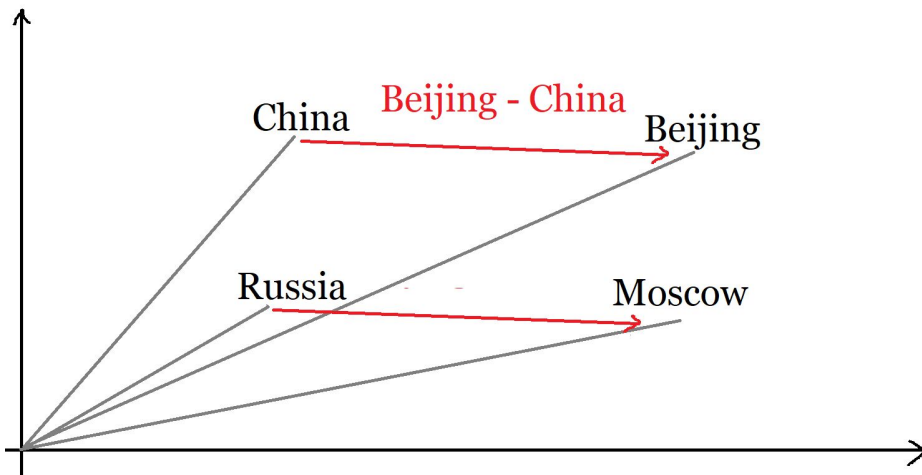
Beijing - China + Russia  $\approx$  Moscow

King - Man + Woman  $\approx$  Queen

## Modelos de Lenguaje Representaciones Continuas (Distribuidas)

Ventajas:

- 2) Composicionalidad aditiva: los word embeddings se distribuyen de manera tal que a veces es posible hacer operaciones algebraicas para obtener analogías



Beijing - China + Russia  $\approx$  Moscow  
King - Man + Woman  $\approx$  Queen

Lecturas recomendadas:

- Analogies Explained: Towards Understanding Word Embeddings: <https://arxiv.org/abs/1901.09813>
- "Analogies Explained" ... Explained: <https://carl-allen.github.io/nlp/2019/07/01/explaining-analogies-explained.html>
- Contrastive Loss is All You Need to Recover Analogies as Parallel Lines: <https://arxiv.org/abs/2306.08221>

## Modelos de Lenguaje Representaciones Continuas (Distribuidas)

Ventajas:

- 3) Mayor generalización: a diferencia de usar IDs, los **word embeddings** proveen información semántica, lo que permite generalizar a combinaciones novedosas de palabras si es que la semántica es relativamente similar a cosas ya vistas en entrenamiento.

Ej: si el modelo maneja el concepto de “**manzana**” y frases como “**Me gusta comer naranjas de postre**”, entonces puede generalizar a frases como “**Me gusta comer manzanas de postre**” aunque nunca la haya visto antes.

## **Modelos de Lenguaje** Representaciones Continuas (Distribuidas)

Otras ventajas:

- Se aprende todo automáticamente (machine learning)
- No se requieren etiquetas al entrenar (entrenamiento **autosupervisado**)
- No hay que estar contando cuántas veces aparece cada posible n-grama
- No estamos limitados a n-gramas (por ejemplo, usando RNNs)

# Contenidos de la clase de hoy

**1 Modelos de Lenguaje (LM)**  
¿Qué son los modelos de lenguaje?  
¿Para qué sirven?  
Modelos de N-gramas

**2 Representaciones Discretas v/s Continuas (Distribuídas)**  
Representaciones Discretas  
Representaciones Continuas (Distribuídas)

**3 Ejemplos concretos**  
Word2Vec  
GloVe  
SkipThought  
Trabajo práctico

# Word2vec (2013)

## Efficient Estimation of Word Representations in Vector Space

**Tomas Mikolov**

Google Inc., Mountain View, CA  
tmikolov@google.com

**Kai Chen**

Google Inc., Mountain View, CA  
kaichen@google.com

**Greg Corrado**

Google Inc., Mountain View, CA  
gcorrado@google.com

**Jeffrey Dean**

Google Inc., Mountain View, CA  
jeff@google.com

## Distributed Representations of Words and Phrases and their Compositionality

**Tomas Mikolov**

Google Inc.  
Mountain View  
mikolov@google.com

**Ilya Sutskever**

Google Inc.  
Mountain View  
ilyasu@google.com

**Kai Chen**

Google Inc.  
Mountain View  
kai@google.com

**Greg Corrado**

Google Inc.  
Mountain View  
gcorrado@google.com

**Jeffrey Dean**

Google Inc.  
Mountain View  
jeff@google.com

<https://papers.nips.cc/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html>

<https://arxiv.org/abs/1301.3781>

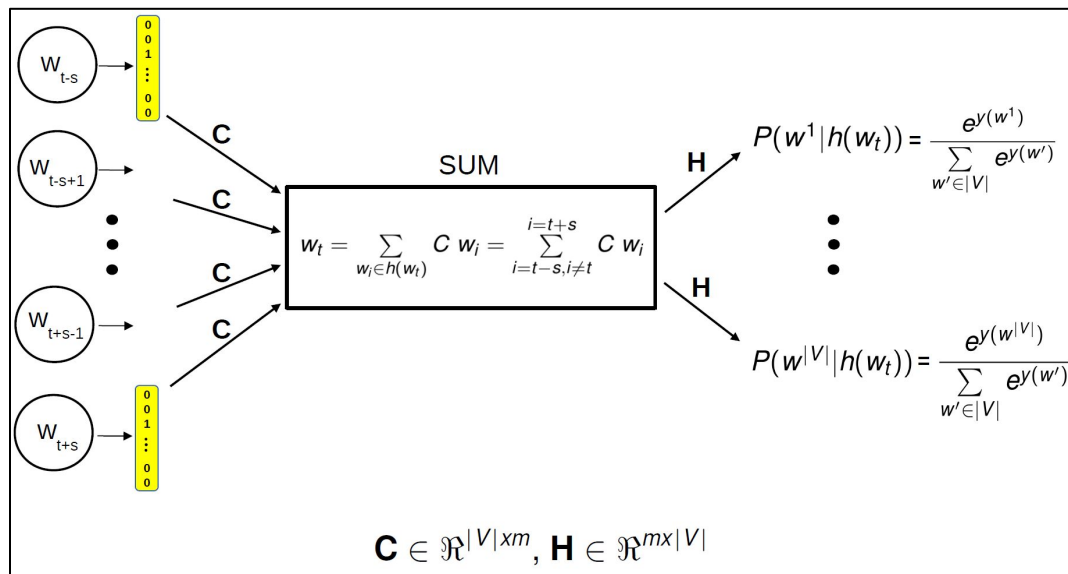
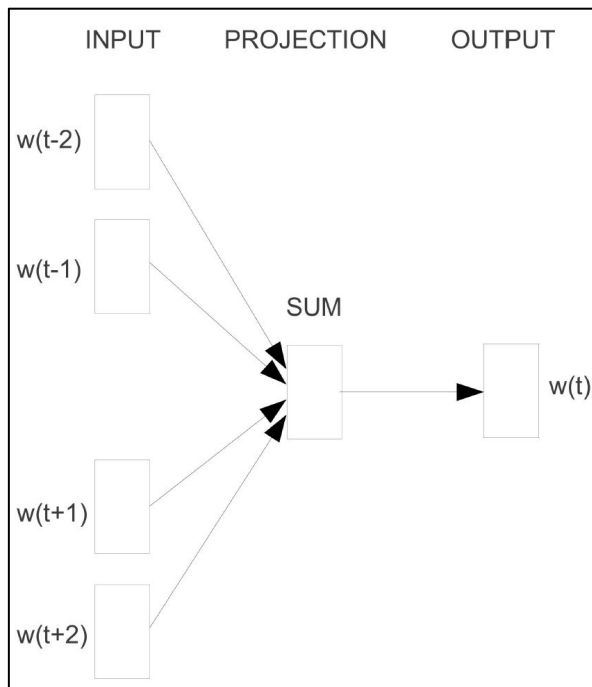
## Word2vec

- Propuesto por Mikolov et al., 2013.
- Propone un modelo simple para aprender **word embeddings** de forma más directa y eficiente.
- Se abandona el objetivo de aprender un modelo de lenguaje: sólo nos interesan los word embeddings (**feature learning**).
- Motivación: **modelos más simples pueden escalar a datasets más grandes.**
- Se proponen dos algoritmos para aprender los word embeddings:
  1. Continuous Bag-of-Words Model (CBoW)
  2. Continuous Skip-gram Model (Skip-gram)



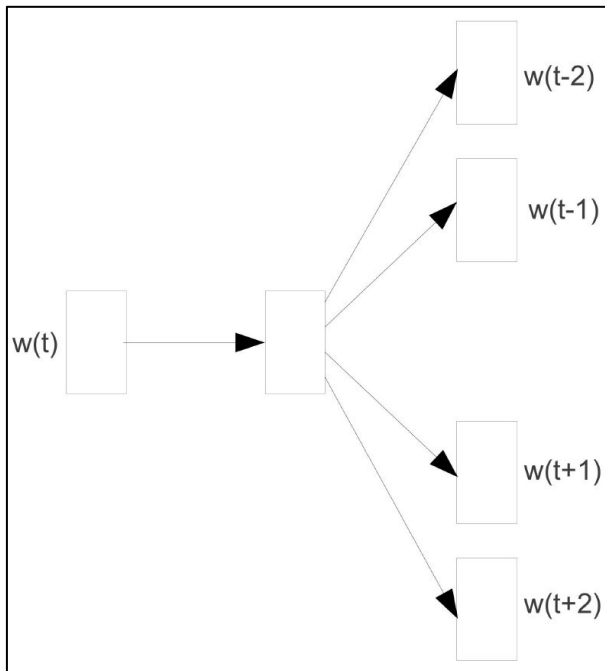
## Word2vec > 1) Continuous Bag-of-Words Model (CBow)

Idea: predecir la palabra de al medio dado el contexto que la rodea



## Word2vec > 2) Continuous Skip-gram Model (Skip-gram)

Idea: predecir el contexto dada la palabra de al medio



The diagram shows the mathematical formulation of the Skip-gram model. It starts with a word  $w_t^i$  in a circle, which points to a yellow vertical bar containing the vector  $\begin{bmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$ . This vector is multiplied by matrix  $\mathbf{C}$  to produce  $w_t = \mathbf{C} w_t^i$  in a box. This box then points to matrix  $\mathbf{H}$ , which leads to the probability equation:

$$P(w^{k \in h(w_t)} | w_t) = \frac{e^{s(w_k, w_t)}}{\sum_{w'_k \in |V|} e^{s(w'_k, w_t)}}$$

where  $s(w_k, w_t) = w_k^T w_t$ .  $\mathbf{C} \in \mathbb{R}^{|V| \times m}$ ,  $\mathbf{H} \in \mathbb{R}^{m \times |V|}$

GloVe (2014)

## **GloVe: Global Vectors for Word Representation**

**Jeffrey Pennington, Richard Socher, Christopher D. Manning**

Computer Science Department, Stanford University, Stanford, CA 94305

`jpennin@stanford.edu, richard@socher.org, manning@stanford.edu`

<https://aclanthology.org/D14-1162.pdf>

## GloVe

- GloVe: Global Vectors for Word Representation
- Propuesto por Pennington et al., 2014
- Similar a Word2vec: se propone un modelo bien sencillo para aprender **word embeddings** (no hay modelo de lenguaje).
- Idea muy general: se busca aprender word embeddings cuyo producto punto aproxime el nivel de co-ocurrencia empírico de las respectivas palabras.

$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

- Para más detalles:
  - link a paper: <https://aclanthology.org/D14-1162.pdf>
  - link a video: <https://youtu.be/ASn7ExxLZws>

## Skip-Thought Vectors (2015)

---

# Skip-Thought Vectors

---

**Ryan Kiros<sup>1</sup>, Yukun Zhu<sup>1</sup>, Ruslan Salakhutdinov<sup>1,2</sup>, Richard S. Zemel<sup>1,2</sup>**

**Antonio Torralba<sup>3</sup>, Raquel Urtasun<sup>1</sup>, Sanja Fidler<sup>1</sup>**

University of Toronto<sup>1</sup>

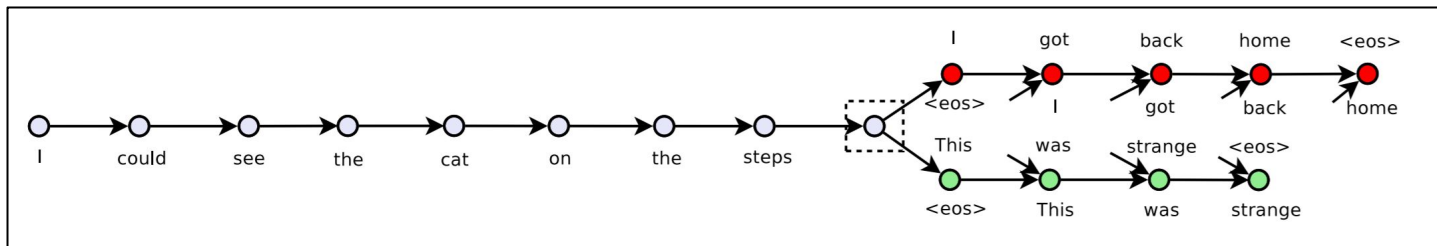
Canadian Institute for Advanced Research<sup>2</sup>

Massachusetts Institute of Technology<sup>3</sup>

<https://papers.nips.cc/paper/2015/file/f442d33fa06832082290ad8544a8da27-Paper.pdf>

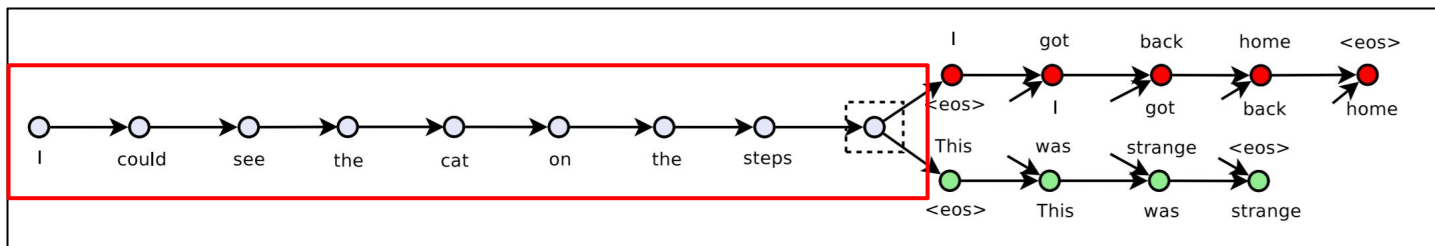
## Skip-Thought Vectors

- Propuesto por Kiros et al., 2015
- Idea: aplicar la misma idea del skip-gram model de Word2vec, pero a nivel de oraciones, para obtener **sentence embeddings**.
- Dada la oración de al medio, predecir la oración anterior y la siguiente.
- Una RNN encoder para la oración de al medio, dos RNN decoders para la anterior y la siguiente.



## Skip-Thought Vectors

- Una vez que se entrena, la RNN encoder se puede usar para obtener **sentence embeddings**, útiles para varias tareas **downstream**, por ejemplo:
  - Semantic relatedness
  - Paraphrase detection
  - Clasificación de oraciones: movie review sentiment, customer product reviews, subjectivity/objectivity classification, opinion polarity, etc.



# Skip-Thought Vectors

| Query and nearest sentence                                                                                                     |                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| he ran his hand inside his coat , double-checking that the unopened letter was still there .                                   | he slipped his hand between his coat and his shirt , where the folded copies lay in a brown envelope .                                   |
| im sure youll have a glamorous evening , she said , giving an exaggerated wink .                                               | im really glad you came to the party tonight , he said , turning to her .                                                                |
| although she could tell he had n't been too invested in any of their other chitchat , he seemed genuinely curious about this . | although he had n't been following her career with a microscope , he 'd definitely taken notice of her appearances .                     |
| an annoying buzz started to ring in my ears , becoming louder and louder as my vision began to swim .                          | a weighty pressure landed on my lungs and my vision blurred at the edges , threatening my consciousness altogether .                     |
| if he had a weapon , he could maybe take out their last imp , and then beat up errol and vanessa .                             | if he could ram them from behind , send them sailing over the far side of the levee , he had a chance of stopping them .                 |
| then , with a stroke of luck , they saw the pair head together towards the portaloos .                                         | then , from out back of the house , they heard a horse scream probably in answer to a pair of sharp spurs digging deep into its flanks . |
| " i 'll take care of it , " goodman said , taking the phonebook .                                                              | " i 'll do that , " julia said , coming in .                                                                                             |
| he finished rolling up scrolls and , placing them to one side , began the more urgent task of finding ale and tankards .       | he righted the table , set the candle on a piece of broken plate , and reached for his flint , steel , and tinder .                      |

Table 2: In each example, the first sentence is a query while the second sentence is its nearest neighbour. Nearest neighbours were scored by cosine similarity from a random sample of 500,000 sentences from our corpus.





Pontificia Universidad Católica de Chile  
Departamento de Ciencias de la Computación

Diplomado en Inteligencia Artificial

## Clase 18

Modelos de lenguaje, Word2Vec, GloVe y SkipThought

Pablo Messina



**IA** Lab

